Implementation roadmap: Weighted Loss TabTransformer + SHAP

The objective is to reproduce the experiments in A Novel Weighted Loss TabTransformer Integrating Explainable AI for Imbalanced Credit Risk Datasets, not just build a loan-prediction demo.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

We'll use PyTorch, the German Credit dataset, class-weighted training and SHAP, then attempt BISAID once its exact data and labels are verified.

PROJECT FINISH LINE

Two datasets, two training strategies, one reproducible comparison

Model

TabTransformer

Experiment

Weighted vs. unweighted loss

Evaluation

Metrics and ROC curves

Explanation

SHAP feature importance

Phase 0: Set up the repository

Do this first. Use Google Colab for convenience or a Python virtual environment on your laptop. The datasets are small, so a GPU is not essential.

Create the project

```
credit-risk-replication/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   ├── 01_data_audit.ipynb
│   ├── 02_preprocessing.ipynb
│   ├── 03_baseline.ipynb
│   ├── 04_weighted_loss.ipynb
│   └── 05_shap_analysis.ipynb
├── src/
│   ├── data.py
│   ├── preprocessing.py
│   ├── model.py
│   ├── train.py
│   ├── evaluate.py
│   └── explain.py
├── configs/
│   ├── german_credit.yaml
│   └── bisaid.yaml
├── outputs/
│   ├── metrics/
│   ├── plots/
│   └── models/
├── requirements.txt
└── README.md
```

Install dependencies

In a terminal:

python -m venv .venv

On Windows, activate it:

.venv\Scripts\activate

Install the libraries:

python -m pip install --upgrade pip

pip install torch pandas numpy scikit-learn matplotlib seaborn shap ucimlrepo tab-transformer-pytorch

Use the existing PyTorch implementation as your starting point: lucidrains/tab-transformer-pytorch. It supports categorical embeddings, transformer layers and continuous features.

GitHub

Done when: imports work and you can create a small TabTransformer and pass a batch through it without errors.

Phase 1: Download and understand German Credit

Start with German Credit, not BISAID. UCI provides 1,000 records with 20 predictive attributes and a binary credit-risk target. It also provides a symbolic version that preserves categorical values.

archive.ics.uci.edu

Dataset: UCI Statlog German Credit Data

Create `01_data_audit.ipynb`

First, load the dataset:

from ucimlrepo import fetch_ucirepodataset = fetch_ucirepo(id=144)X = dataset.data.features.copy()y = dataset.data.targets.iloc[:, 0].copy()print("Features:", X.shape)print("Target shape:", y.shape)print("\nFeature types:")print(X.dtypes)print("\nTarget distribution:")print(y.value_counts(dropna=False))print("\nMissing values:")print(X.isna().sum())

If the downloaded target uses the standard UCI labels, map it to the labels used in the paper:

Standard UCI coding: 1 = good, 2 = bad# Paper coding: 0 = bad, 1 = goody = y.astype(int).map({2: 0, 1: 1})assert y.notna().all()assert set(y.unique()) == {0, 1}

Check the class distribution. The paper describes 300 bad-credit and 700 good-credit records, with bad credit represented by class 0 and good credit by class 1.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Also inspect the feature definitions from UCI. Some attributes are categorical even when represented by numbers, so do not classify them as continuous using data type alone.

Save these outputs: dataset dimensions, feature names, data types, missing-value counts, class counts and the target-label mapping.

Done when: you can explain every feature and confirm that your target labels match the paper.

Phase 2: Split the data and preprocess it

Create `02_preprocessing.ipynb` and implement the 80:10:10 train, validation and test split described in the paper.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf
from sklearn.model_selection import train_test_splitX_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.20, stratify=y, random_state=42)X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=42)print(X_train.shape, X_val.shape, X_test.shape)

This gives approximately 800 training, 100 validation and 100 test records.

Now preprocess the features.

Feature type	What to do
Categorical	Convert each category to an integer ID for its embedding
Numerical	Fit a `MinMaxScaler(feature_range=(-1, 1))` on training data
Missing values	Fit an appropriate imputer on training data
Unseen categories	Reserve an unknown-category ID

Target	Preserve the paper's class mapping

Important: the paper mentions one-hot encoding, but TabTransformer uses categorical embeddings in its architecture. Those are different representations. For the main implementation, use integer category IDs and embeddings, and record this methodological ambiguity. Do not silently pretend the preprocessing is an exact match.

Done when: all features are model-ready, the class proportions are sensible across splits, and no preprocessing step learns from validation or test records.

Phase 3: Implement TabTransformer

Create `src/model.py`.

You can use the existing implementation instead of writing attention from scratch. The important part is to instantiate it with the paper's reported hyperparameters and understand its inputs and output.

GitHub

Configuration to use

Setting	German Credit	BISAID
Embedding dimension	64	128
Transformer depth	2	1
Attention heads	2	2
Attention dropout	0.1	0.1

Feed-forward dropout	0.1	0.1
Output classes	2	4
Batch size	64	64

These settings come from the paper's Table 4 and training discussion.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Use this basic model structure for German Credit:

import torch.nn as nnfrom tab_transformer_pytorch import TabTransformermodel = TabTransformer(categories=tuple(category_cardinalities), num_continuous=len(numerical_columns), dim=64, dim_out=2, depth=2, heads=2, attn_dropout=0.1, ff_dropout=0.1)criterion = nn.CrossEntropyLoss()

Here, `category_cardinalities` contains the number of possible encoded values for each categorical column. `numerical_columns` contains your numerical feature names.

Your batch must provide categorical features as integer tensors and numerical features as floating-point tensors. The model produces two class scores per record. Pass those raw scores, called logits, to `CrossEntropyLoss`.

Some training details, including optimizer and learning rate, are not fully specified in the paper. For your first run, use AdamW with a learning rate of `0.001`, up to 100 epochs, and save the checkpoint with the best validation loss. Label these as your implementation choices, not as confirmed author settings.

Done when: one training batch runs successfully, produces two scores per record, and returns a finite loss.

Phase 4: Train the unweighted baseline

Create `src/train.py`.

Implement a standard PyTorch training loop:

1. Send each training batch through the model.

2. Calculate ordinary cross-entropy loss.
3. Clear old gradients.
4. Backpropagate the loss.
5. Update parameters using the optimizer.
6. Calculate validation loss after each epoch.
7. Save the best validation checkpoint and training history.

Keep the initial run reproducible:

import randomimport numpy as npimport torchSEED = 42random.seed(SEED)np.random.seed(SEED)torch.manual_seed(SEED)if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)

After training, save the model checkpoint and validation-loss plot.

Also train a logistic regression model as a quick sanity-check baseline. It is not the paper's central experiment, but it helps identify problems with your data pipeline.

Done when: the unweighted TabTransformer finishes training and generates predictions on the test set.

Phase 5: Add the weighted loss

This is the central experiment.

The paper defines the class weight as

$$w_i = \frac{N}{N_i}$$

where N is the total number of samples and N_i is the number of samples in class i.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Create `src/losses.py`:

import numpy as npimport torchimport torch.nn as nndef make_weighted_loss(y_for_weights, num_classes, device): counts = np.bincount(np.asarray(y_for_weights, dtype=int), minlength=num_classes) if np.any(counts == 0): raise ValueError("Every class needs at least one sample.") weights = len(y_for_weights) / counts weights = torch.tensor(weights, dtype=torch.float32, device=device) return nn.CrossEntropyLoss(weight=weights)

Use it in a second training run:

criterion = make_weighted_loss(y_train, num_classes=2, device=device)

The model architecture and training settings should otherwise remain unchanged.

Run three experiments

Experiment	Loss	Purpose
A	Ordinary cross-entropy	Baseline
B	Weighted loss calculated from the full dataset's class counts	Reproduce the paper's reported weighting procedure
C	Weighted loss calculated from training labels only	Leakage-conscious comparison

Experiment B follows the paper's stated use of the original dataset distribution before splitting. Experiment C is the more conservative approach because it does not use held-out labels when calculating weights. Report them separately.

For every experiment, use the same split, preprocessing, training setup and evaluation code.

Done when: you have two working TabTransformer checkpoints, plus the optional third run, and a results file identifying each configuration.

Phase 6: Reproduce the evaluation tables

Create `src/evaluate.py`.

Calculate these metrics for every model:

- Accuracy.
- Precision, recall and F1 score for each class.
- Macro-F1 and macro-recall.
- F2 score, because the paper discusses it in the methods.
- Confusion matrix.
- ROC curve and AUC, using one-vs-rest for the four-class problem.

The paper reports the following accuracy figures as reference targets.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Dataset	Unweighted	Weighted
German Credit	93%	95%
BISAID	86.35%	89.27%

Do not expect an exact match automatically. The paper has some methodological ambiguities, and random initialization, preprocessing and split choices can all affect performance.

Also, do not select your model or tune hyperparameters based on test results. Use validation data for those decisions and touch the test set only for final evaluation.

After the first successful run

The paper mentions ten-fold cross-validation during hyperparameter tuning.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Add this as a second experiment stage:

- Keep the final test set untouched.
- Perform tuning using only the training data, with stratified folds.
- Record the parameter search space.
- Select the configuration using cross-validation.
- Retrain with the selected settings and evaluate once on the test set.

The exact search grid is not fully documented in the paper, so your search space must be reported as your own implementation choice.

Phase 7: Implement SHAP

Create `src/explain.py`.

Once the model comparison works, explain the predictions of your final weighted model using the SHAP library. SHAP provides a common explanation interface that can work with a prediction function.

SHAP latest documentation
+1

Your implementation should:

1. Create a prediction wrapper that accepts the original feature columns and internally applies the saved preprocessing.
2. Return the model's class probabilities.
3. Select a small background sample from training data.
4. Explain held-out records using a suitable model-agnostic SHAP explainer.
5. Calculate mean absolute SHAP importance for each original feature.
6. Save global feature-importance plots and a few individual prediction explanations.

The most important results to compare with the paper are the reported feature rankings:

Dataset	Features emphasized in the paper
German Credit	Credit amount, age, existing credit status
BISAID	Financing needs, annual sales, total assets

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Your SHAP values may differ even when your model performs similarly. Use consistent background data and explain which class's probability you are interpreting. SHAP describes the model's behavior; it does not prove that a feature causes an outcome.

Done when: you have a global importance chart and individual explanations that you can describe in plain English.

Phase 8: Add BISAID

Do this only after German Credit is working end to end.

The paper reports 2,563 BISAID records, 12 original features, five removed identity-related fields and four target classes. The reported class counts are 379, 1,146, 539 and 499.

A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf
A_Novel_Weighted_Loss_TabTransformer_Integrating_Explainable_AI_for_Imbalanced_Credit_Risk_Datasets.pdf

Before training, verify the following:

- You have access to the original data or a demonstrably equivalent snapshot.
- You have confirmed the meaning of all four target classes.
- You know exactly which columns were removed and why.
- Your numerical and categorical feature assignments match the source data.
- Your split and class-weight calculations are documented.

The paper does not make the class semantics sufficiently clear to reconstruct them confidently from its text alone. Consult the dataset documentation or contact the corresponding author if necessary. Do not invent target definitions simply to make the code run.

Once verified, reuse the same pipeline with the BISAID configuration from Phase 3.

Phase 9: Package your final results

Your repository should make it possible for another student to run the experiments and understand where every reported number came from.

Save these artifacts:

Artifact	File
Dataset and preprocessing audit	`outputs/data_audit.json`
Experiment configurations	`configs/`
Evaluation metrics	`outputs/metrics/results.csv`

Confusion matrices and ROC curves	`outputs/plots/`
Training and validation histories	`outputs/plots/loss_curves.png`
SHAP plots	`outputs/plots/shap/`
Final model checkpoints	`outputs/models/`
Replication notes and discrepancies	`REPRODUCIBILITY_NOTES.md`

Your README should explain how to install the dependencies, obtain the data, run each experiment, and reproduce the evaluation. Do not commit restricted data, personal information, or unnecessarily large checkpoints.

Your execution checklist

Project tracker

0 of 9 complete

Create the repository and Python environment

Install PyTorch, pandas, scikit-learn, SHAP and the TabTransformer implementation.

Load and audit German Credit

Verify 1,000 records, feature types, target mapping and class counts.

Implement the 80:10:10 split and preprocessing

Fit transforms on training data only and save the fitted objects.

Build and train the unweighted TabTransformer

Match the reported German Credit architecture settings.

Train the weighted version

Keep everything fixed except the loss weighting.

Generate evaluation tables and ROC curves

Compare accuracy and per-class metrics.

Generate SHAP explanations

Produce global feature importance and individual explanations.

Verify BISAID data and labels

Proceed only when the data and four-class target mapping are confirmed.

Document discrepancies and reproduce the final run

Record settings, seeds, results and limitations.

Copy checklist

Start today with only the first three phases: set up the environment, audit German Credit, and prepare the data. Do not start building a dashboard, tuning ten different models, or worrying about BISAID until the first unweighted TabTransformer trains successfully. That gives you a solid, testable foundation for the entire replication.